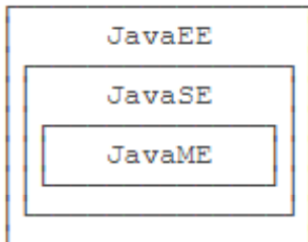


Lecture 11

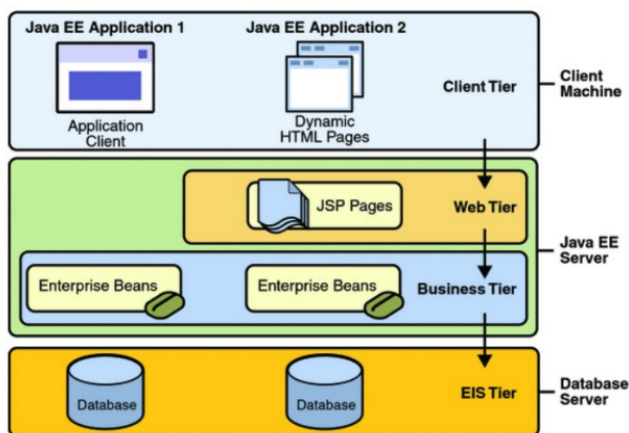
- JavaEE

- Java EE（企业级 Java 平台）

- Java EE（企业版 Java）概述：



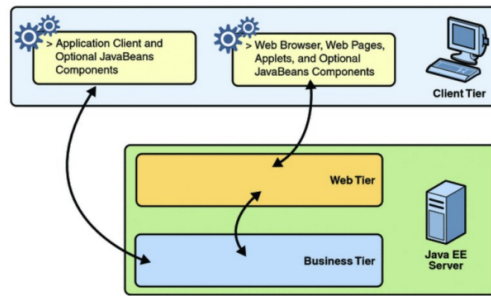
- 用于创建 Web 应用的 Java 技术属于 Java EE 平台的一部分
 - Java EE 构建于 Java SE（标准版）之上，Java SE 提供我们日常使用的核心 API（如 java.lang、java.io 等）
 - Java EE 在此基础上加了数据库访问（JDBC、JPA）、Servlet、远程方法调用（RMI）、消息（JMS）、Web 服务、XML 处理、企业级组件（Enterprise Beans）等库
 - Java EE 提供 API 与运行时环境，帮助开发者构建大规模、多层次、可扩展、可靠且安全的 Web/业务应用
 - 多层（Multitiered）应用：



- Java EE 通过多层应用模型减少企业应用的复杂性。这种相互隔离的层（tiers）有助于职责分离、可维护性和可扩展性
 - 客户端层 Client Tier：由向中间层发出请求的客户端程序组成
 - Java EE 客户端可以是网页客户端或应用程序客户端

Application client

- runs on a client machine and typically has a GUI (e.g., created from Swing or AWT)
- Can directly access enterprise beans running in the business tier or communicate with a servlet running in the web tier
- Can be written in other languages and interact with Java EE servers



Web client

- consists of web pages and a web browser
- usually do not query databases, execute complex business rules
- A web page received from the web tier can include an embedded applet, a small client application written in Java that executes in JVM installed in the web browser

应用程序客户端：

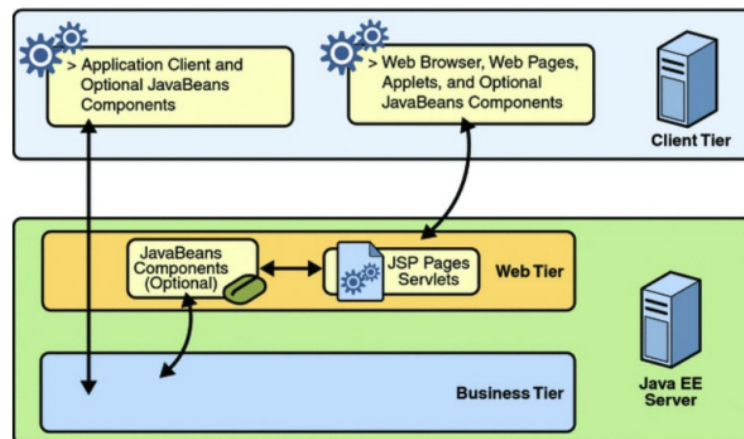
- 在客户机上运行，通常具有图形界面（例如使用 Swing 或 AWT 创建）
- 可直接访问在业务层运行的企业组件，或与运行在 Web 层的 servlet 通信
- 可用其他语言编写并与 Java EE 服务器交互

网页客户端：

- 由网页和浏览器组成
- 通常不直接查询数据库或执行业务复杂规则
- 从 Web 层接收的网页可包含嵌入的小型客户端程序（例如在浏览器 JVM 中运行的 applet）

中间层：

- Web 层（处理 Web 请求、动态页面如 JSP）
 - Web 层由处理客户端与业务层交互的组件组成

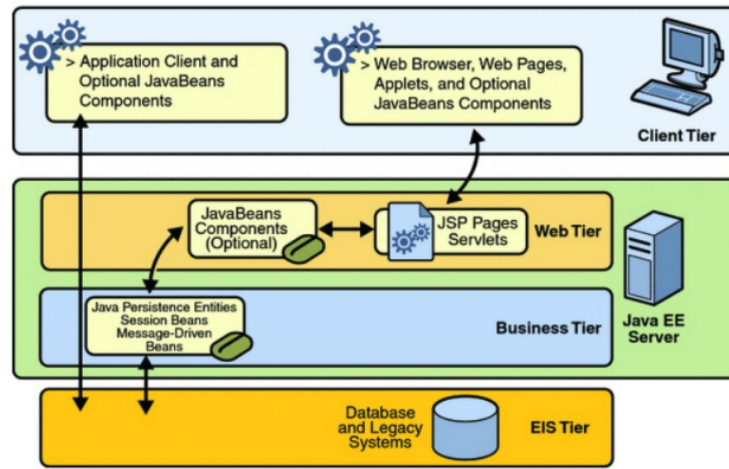


Web 层常用技术：

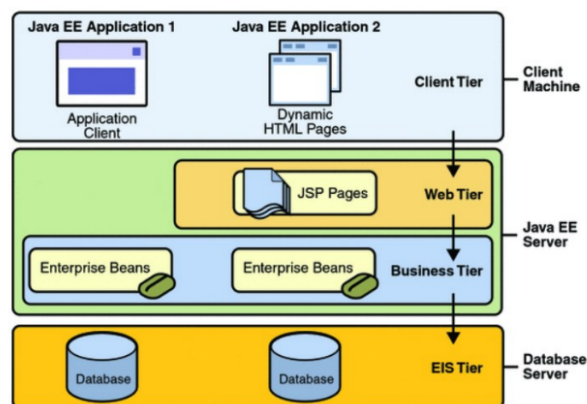
- Servlet：用于动态处理请求并构建响应的服务器端 Java 类（API）
- JSP（服务器端页面）：扩展/执行 Servlet，通过生成包含 HTML、XML 等的网页来实现用户界面

业务层（处理客户端请求并执行业务逻辑） Business Tier

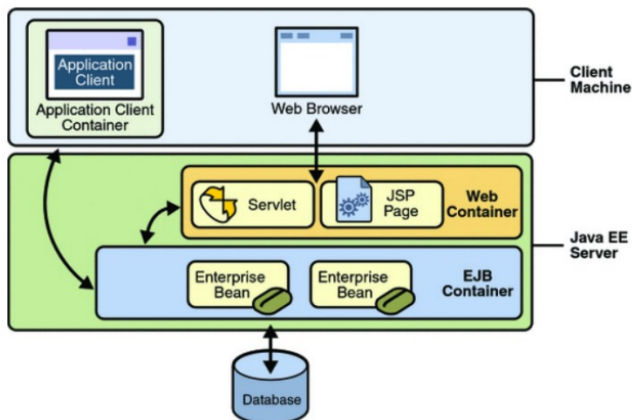
- 业务层负责解决或满足特定业务领域（如银行、零售、金融）的业务需求



- 业务逻辑通常由在业务层运行的企业组件（Enterprise Beans）来实现
- 在设计良好的企业应用中，核心功能应放在业务层组件中
- 企业层技术（示例）：
 - 企业级组件（如企业 JavaBean），提供事务管理、远程调用、消息驱动、安全、生命周期管理和负载均衡等特性，便于实现企业级开发
- 数据层：在永久性数据存储（数据库）中保存和管理应用数据（企业信息系统层）
 - 又称企业信息系统（EIS）层



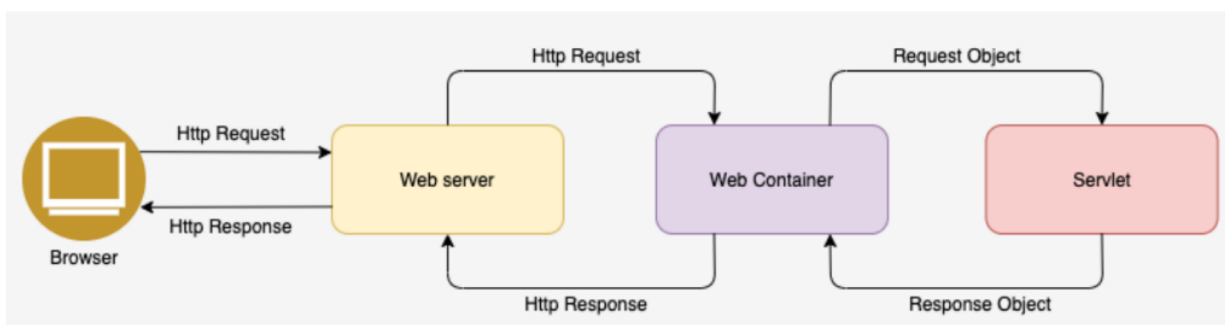
- EIS 包含数据库服务器、企业资源计划系统及其他遗留数据源，通常位于与 Java EE 服务器不同的机器上，由业务层访问
- 数据层常用技术：
 - Java 数据库连接 API（JDBC）
 - Java 持久化 API（JPA）
 - Java 事务 API（JTA）
- Java EE 服务器承载多种应用组件类型（例如 servlet、企业组件），对应多层应用中的各个层
- Java EE 服务器通过“容器”为这些组件提供运行时服务
- 容器提供标准化的运行环境，包含并发管理、生命周期管理、请求处理等服务



- 在执行网页组件、企业组件或应用客户端组件前，必须将其打包成 Java EE 模块并部署到相应的容器中
- 企业 JavaBean 容器：管理企业组件的执行，企业组件及其容器在 Java EE 服务器上运行
- Web 容器：管理 JSP 页面和 servlet 组件的执行，Web 组件及其容器在 Java EE 服务器上运行
- 应用客户端容器：管理应用客户端组件的执行，应用客户端及其容器在客户端上运行
- Applet 容器：管理小程序（applet）的执行，通常由浏览器与 Java 插件共同提供运行环境

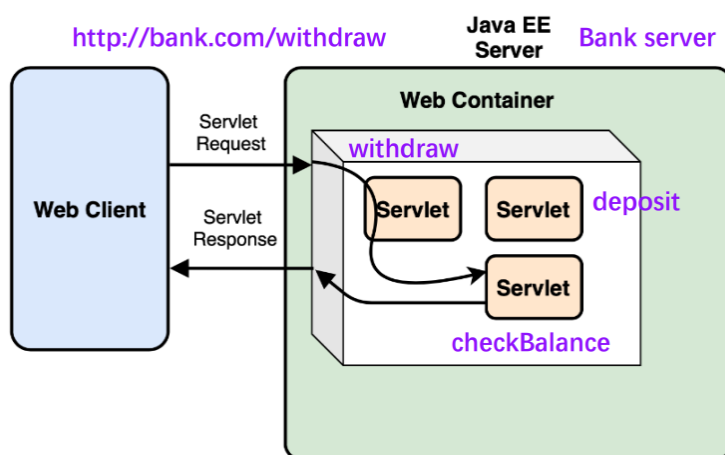
• Servlet 与容器

- Servlet 本质上是一个 Java 程序/类
- Servlet 通过实现应用或业务逻辑来响应传入请求
- Servlet 无法直接理解原始的 HTTP 请求（字节流）；它作为 Java 程序，处理的是对象
- Servlet 在 servlet 容器中运行，容器提供标准化运行时服务，如请求处理、生命周期管理和并发控制
- workflow :



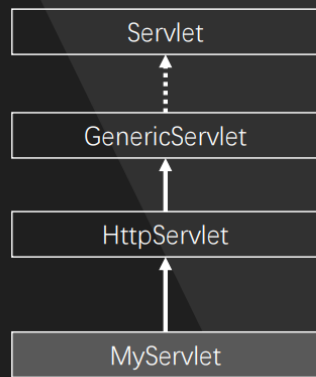
- 客户端向 Web 服务器发送 HTTP 请求，Web 服务器将请求转发给 Web 容器
- Web 容器将 HTTP 请求解析为对象，并将请求对象转发给相应的 Servlet
- Servlet 执行应用逻辑，构建响应对象并返回给 Web 容器
- Web 容器将响应对象转换为等效的 HTTP 响应，并发送给 Web 服务器
- Web 服务器通过 HTTP 响应将结果返回给客户端

- Web 服务器
 - 通过 HTTP 协议提供静态文件（例如 HTML、图片、文本）
 - 可以用几行 Java 代码实现一个非常简单的 Web 服务器，或使用开源服务器（例如 Apache HTTPD）
- Web 容器
 - 通过执行服务器端的 Web 组件（例如 Servlet）来提供动态内容
 - 将 HTTP 请求转换为请求对象，并将响应对象转换为 HTTP 响应
- 容器与 Servlet 工作方式（将 URL 路径映射到对应的 Servlet，通常通过 web.xml 或注解完成）



- 示例流程：
 - 客户端访问特定 URL（例如 <http://bank.com/withdraw>）
 - Web 容器接收请求并根据映射找到对应的 Servlet（如 withdraw、deposit、checkBalance）
 - 将请求封装为请求对象并传递给相应的 Servlet 处理
 - Servlet 生成响应对象，返回给容器，由容器转换为 HTTP 响应并发回客户端
- Servlet 接口（定义所有 Servlet 必须实现的方法）
 - `init(ServletConfig config)`：容器调用，初始化 Servlet 并传入启动参数
 - `service(ServletRequest req, ServletResponse res)`：容器调用，允许 Servlet 响应请求并处理业务逻辑
 - `getServletConfig()`：返回包含初始化和启动参数的 `ServletConfig` 对象
 - `getServletInfo()`：返回 Servlet 的基本信息（例如作者、版本、版权等）
 - `destroy()`：容器调用，指示 Servlet 即将被移除或停止服务
 - 这些方法由容器在适当时机调用，用于初始化、处理请求和销毁 Servlet

The HttpServlet Class



- GenericServlet 实现了 Servlet
 - 抽象类，通用的、与协议无关的 Servlet 基类
- HttpServlet 继承自 GenericServlet
 - 抽象类，定义了针对 HTTP 协议的 Servlet 行为
 - 通常直接继承 HttpServlet 来创建自定义的 HTTP servlet
 - HttpServlet 常见方法（可被重写）：
 - doGet、doPost、doPut、doDelete、doHead、doOptions、doTrace
 - service(HttpServletRequest, HttpServletResponse) 用于接收并分派请求
 - getLastModified、getServletInfo、getServletConfig、destroy 辅助方法
 - 定义了若干表示 HTTP 方法名的常量：METHOD_GET、METHOD_POST、METHOD_PUT、METHOD_DELETE、METHOD_HEAD、METHOD_OPTIONS、METHOD_TRACE 等

```
protected void service(HttpServletRequest req, HttpServletResponse resp) {
    String method = req.getMethod();
    long lastModified;
    if (method.equals("GET")) {...} else if (method.equals("POST")) {
        lastModified = this.getLastModified(req);
        this.maybeSetLastModified(resp, lastModified);
        this.doHead(req, resp);
    } else if (method.equals("PUT")) {
        this.doPost(req, resp);
    } else if (method.equals("DELETE")) {
        this.doPut(req, resp);
    } else if (method.equals("TRACE")) {
        this.doDelete(req, resp);
    } else if (method.equals("OPTIONS")) {
        this.doOptions(req, resp);
    }
}
```

- 提供一个可被子类化的抽象基类（HttpServlet）
- HttpServlet 覆盖了 service(), 将 HTTP 请求按方法类型分派到对应的处理方法（例如 GET -> doGet()）

- HttpServlet 的子类至少应重写一个处理方法，通常是：
 - doGet 由容器（通过 service 方法）调用以处理 GET 请求

```
@WebServlet(name = "helloServlet", value = "/hello-servlet")
public class HelloServlet extends HttpServlet {
    2 usages
    private String message;                                init() and destroy() manage resources
                                                            that are held for the life of the servlet

    public void init() { message = "Hello World!"; }

    public void doGet(HttpServletRequest request, HttpServletResponse response)
        response.setContentType("text/html");

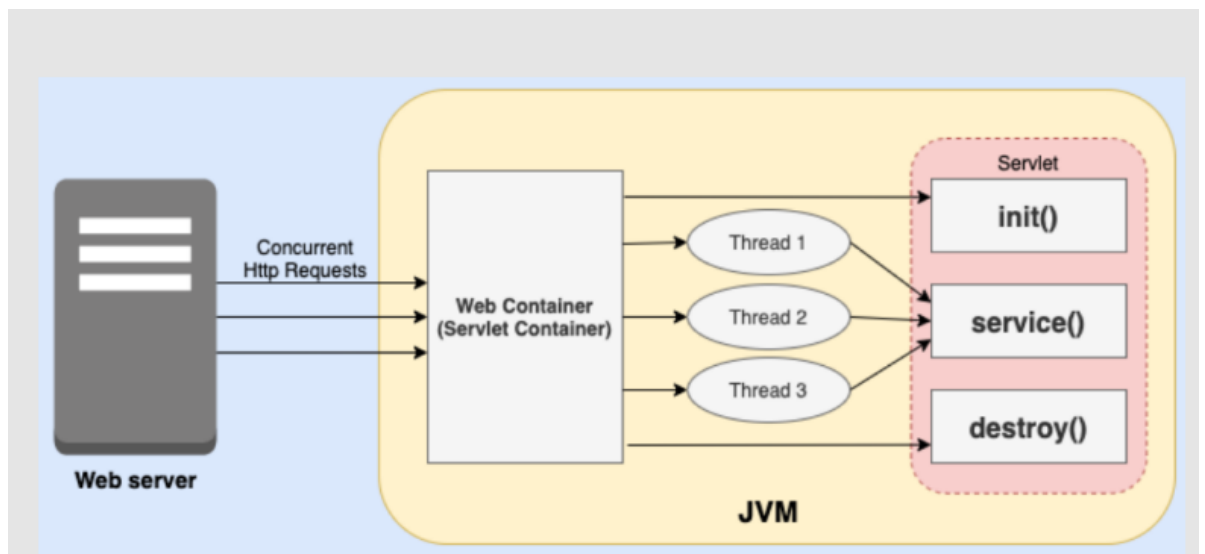
        // Hello
        PrintWriter out = response.getWriter();
        out.println("<html><body>");
        out.println("<h1>" + message + "</h1>");
        out.println("</body></html>");
    }

    public void destroy() {
    }
}
```

@WebServlet annotation is used to declare a servlet.
The annotated class must extend the HttpServlet class.

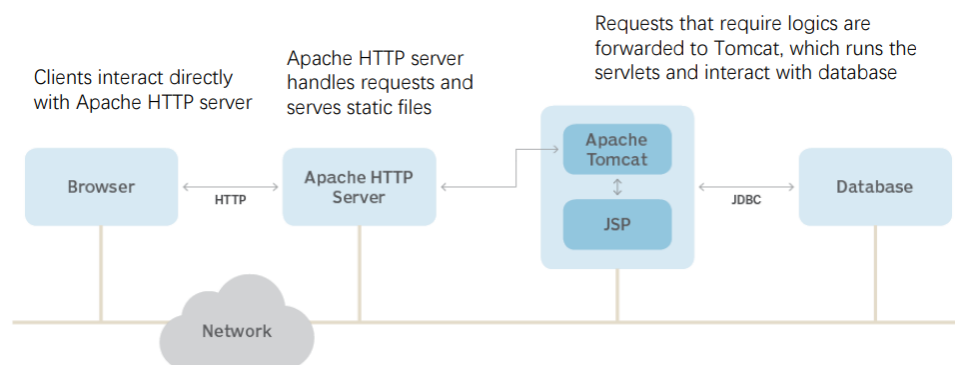
- value: required, specify the URL of this servlet
- Name: optional, specify the name of this servlet

- 覆盖 doGet 时的典型步骤：
 - 读取请求数据（如请求参数）
 - 设置响应头（例如 Content-Type、字符编码）
 - 获取响应的 Writer 或输出流（response.getWriter() / response.getOutputStream()）
 - 将响应数据写出并完成返回
 - 建议在响应中明确指定内容类型和编码，确保浏览器正确显示
 - doPost（处理 HTTP POST 请求）
 - doPut（处理 HTTP PUT 请求）
 - doDelete（处理 HTTP DELETE 请求）
 - 以及 init 与 destroy 方法用于管理 servlet 生命周期期间持有的资源
- Servlet 容器：
 - Servlet 容器就是在 JVM 之上运行的已编译可执行程序
 - 容器的主要功能是加载、初始化并执行 servlet
 - Servlet 容器管理 servlet 的整个生命周期



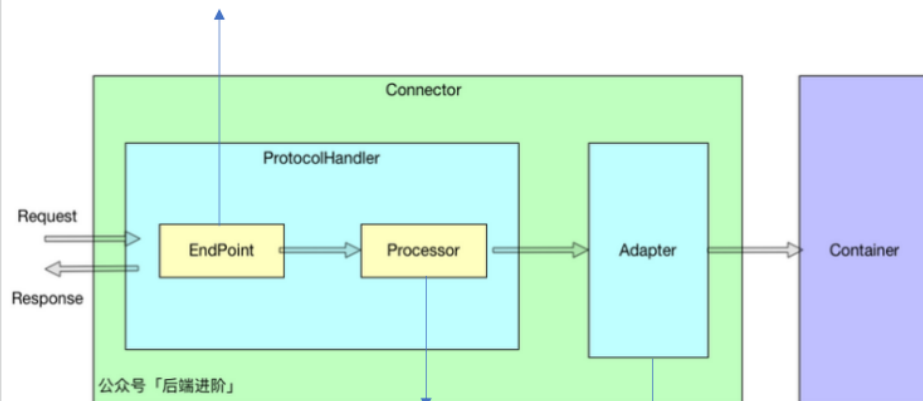
- 并发的 HTTP 请求被转发到 Web 容器
- Web 容器创建 servlet 实例并调用 init()（仅调用一次）
- 容器通过产生多条线程来处理对同一实例的多个请求，每个线程执行该实例的 service() 方法
- 当所有线程结束后，容器调用 destroy() 并将 servlet 实例从容器中移除
- Multithreading:
 - Java servlet 容器通常是多线程的：同一 servlet 可以同时处理多个请求
 - 容器负责多线程调度与并发执行
 - 默认情况下，每个 servlet 声明通常只有一个实例
 - 容器通过在不同线程上并发执行 service 方法来处理并发请求
 - 应用开发者需确保 servlet 代码是线程安全的（注意访问共享资源，如实例变量/类变量）
- java .servlet
 - 过去我们主要使用 Java SE JDK
 - javax.servlet 属于 Java EE，通常随 Java EE SDK 提供
 - 或者，像 Tomcat 这样的轻量级 servlet 容器也会自带该 API（例如 servlet-api.jar）
- Java EE：
 - Java EE 是一套抽象规范，描述了标准、期望的行为以及 API 之间的交互
 - 包括具体的 API（如 Servlet 接口）
 - 也包括自然语言的规范（如如何管理 servlet 的生命周期）
 - 任何公司或开发者都可以实现该规范并提供具体可用的实现
 - 如果一个应用满足 Java EE 规范的要求，则称该应用为“符合 Java EE”的实现
 - 在 Java 规范中，通常会在起草规范时提供一个参考实现（Reference Implementation, RI）
 - 其他厂商可以基于规范开发自己的实现（通常宣称在某些方面更优）

- Java EE 开发者应按规定编写代码（导入 javax 等接口），这样代码即可在任意符合规范的实现上正确运行
- Apache Tomcat
 - Apache Tomcat 是一个 Web 容器，可运行基于 Servlet 和 JSP 的 Web 应用



- Tomcat 用 Java 编写，需要 JDK 运行
- 工作方式（示意）
 - 专门的 Web 服务器（如 Apache HTTP）可负责静态内容和请求分发
 - 需要业务逻辑的请求可转发给 Tomcat，由其运行 servlet 并与数据库交互
- 特点与建议
 - Tomcat 自带简单的 HTTP 服务器，能直接提供静态内容
 - 但作为通用 HTTP 服务器，其静态内容处理性能通常不及专用的 Web 服务器（如 Apache HTTP）
 - 对于简单的生产环境应用，单独使用 Tomcat 往往已足够且效果良好
- Tomcat 的架构要点：
 - Tomcat 表示一个服务器实例
 - 一个服务器可提供多个服务（不同端口）
 - 一个服务可以包含多个连接器（Connector）
 - 连接器负责处理网络连接
 - 不同连接器可支持不同的网络协议
 - 每个服务只包含一个容器/引擎（Engine），负责内部 Servlet 的处理
 - 连接器通过 ServletRequest 与 ServletResponse 对象与容器通信
- Connector（连接器）职责：

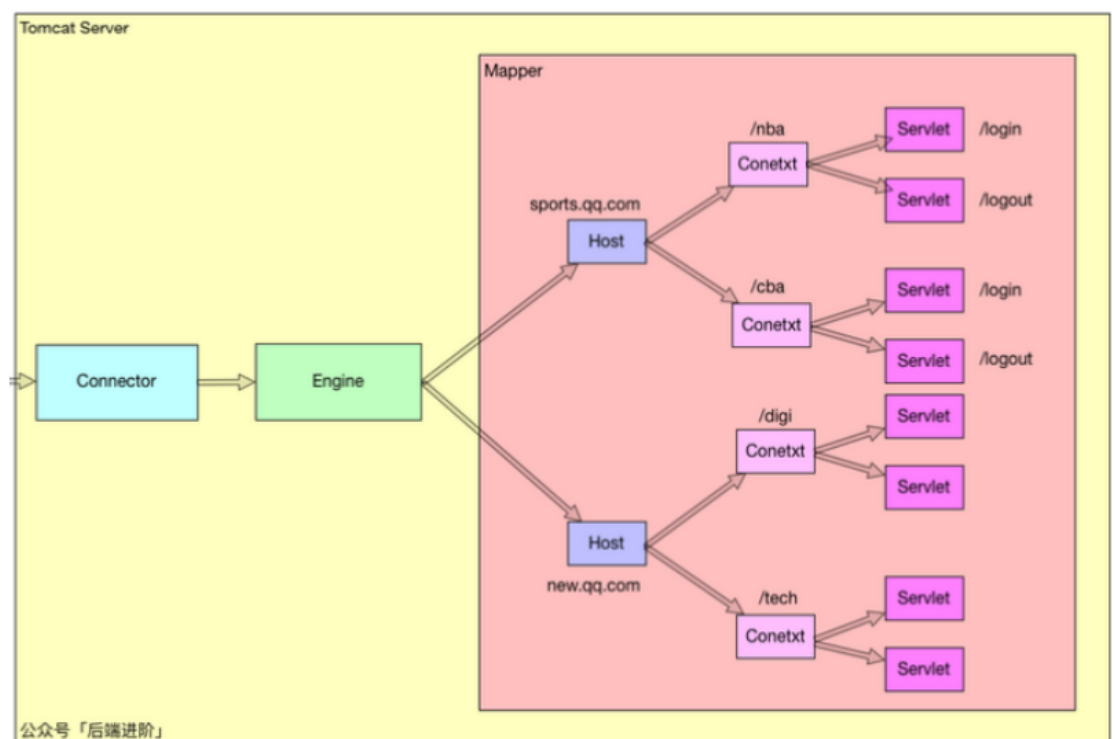
Endpoint components handle underlying Socket network connections



Processor encapsulates the Socket byte stream into a Request object

Adapter adapts the Request object to the Request object that the container can recognize

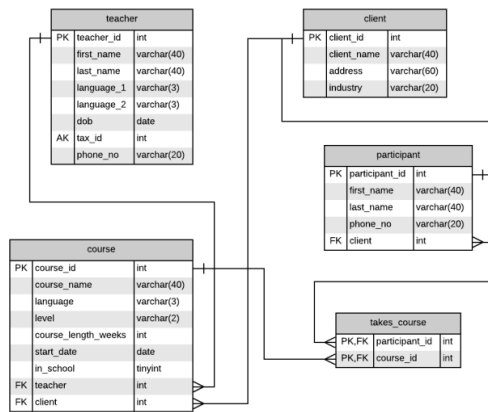
- 封装各种网络协议，屏蔽网络连接和 I/O 处理的细节
- 将底层 Socket 字节流处理成请求对象（Request）并传递给容器
- Tomcat 将处理请求的细节封装到 ProtocolHandler 接口上
- 连接器内部通常包含 EndPoint（端点）、Processor（处理器）与 Adapter（适配器）：
 - Processor 将 Socket 字节流封装为 Request 对象
 - Adapter 将 Request 对象适配为容器可识别的请求格式
- Container（容器 / 引擎）概念与职责：



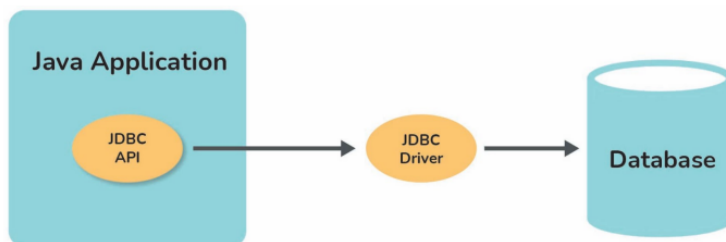
- Engine:
 - 代表与某个 Service 关联的全部请求处理机制
 - 连接器接收到的所有请求先交给 Engine，再由 Engine 转发到对应的虚拟主机 (Host)
 - 容器内部通常按层次组织 (Engine → Host → Context → Wrapper → Servlet)，负责映射 URL 到具体 Servlet 并执行
- Host (主机 / 虚拟主机) 要点:
 - Host 表示一个虚拟主机
 - 一个 Engine 可以包含多个虚拟主机
 - 每个虚拟主机有自己的域名 (用于区分不同站点/上下文)
- Context (应用上下文)
 - 表示一个应用程序 (Context)
 - 一个虚拟主机 (Host) 可以包含多个应用
 - 每个应用可以配置多个 Servlet (多个上下文路径/模块)
- Wrapper (封装器)
 - 表示单个 Servlet 的封装单元
 - 每个 Wrapper 关联并管理一个具体的 Servlet 实例 (负责生命周期、调用等)
- Mapper (映射器)
 - 根据请求的 URL 定位对应的 Host (虚拟主机) 和 Context (应用)
 - 使用 web.xml 或注解等配置定位并映射到具体的 Servlet

• JDBC 与 JPA (数据库访问与对象关系映射)

- 文件系统与数据库比较:
 - 文件系统存储非结构化、无关的数据，适用场景:
 - 需要对数据做版本控制 (数据库处理不便)
 - 有大块且经常增长的数据 (例如日志文件)
 - 需要让其他应用直接访问数据而无需 API (例如文本编辑器)
 - 存储大量二进制内容 (如图片、音频)
 - 数据库存储相关的结构化数据，便于高效的插入、更新与检索，适用场景:
 - 需要存储大量结构相同的记录行
 - 需要极快的查找与查询处理
 - 需要支持多用户并发与原子性事务 (数据安全)
- 关系型数据库:



- 将数据组织为行与列，集合起来形成表格
- 数据通常分布在多个表中，通过主键或外键进行关联与连接
- 主键/外键等唯一标识展示表之间存在的各种关系
- JDBC（Java 数据库连接）API
 - 大多数应用为了存储、组织与检索数据使用关系型数据库
 - Java EE 应用通过 JDBC API 访问关系型数据库
 - JDBC API 位于 java.sql 与 javax.sql 包中
 - JDBC 是一套标准 Java 接口/类，定义了 Java 程序如何与数据库通信
 - 关键接口：Connection、Statement、ResultSet 等
 - 提供与数据库无关的方式执行 SQL（例如 executeQuery(...)）
- JDBC 驱动：



- JDBC 驱动是 JDBC API 的数据库特定实现
- 由数据库厂商提供（如 MySQL、PostgreSQL 等）
- 将 Java 的调用翻译为数据库特定协议，管理与数据库服务器的底层通信
- 需要在客户端机器上安装相应的 JDBC 驱动程序
-

Example of Using JDBC <https://docs.oracle.com/javase/tutorial/jdbc/TOC.html>

```
public Connection getConnection() throws SQLException {
    Connection conn = null;
    Properties connectionProps = new Properties();
    connectionProps.put("user", this.userName);
    connectionProps.put("password", this.password);

    if (this.dms.equals("mysql")) {
        conn = DriverManager.getConnection(
            "jdbc:" + this.dms + "://" +
            this.serverName +
            ":" + this.portNumber + "/" +
            connectionProps);
    } else if (this.dms.equals("derby")) {
        conn = DriverManager.getConnection(
            "jdbc:" + this.dms + "://" +
            this.dbName +
            ";create=true",
            connectionProps);
    }
    System.out.println("Connected to database");
    return conn;
}

public static void viewTable(Connection con) throws SQLException {
    String query = "select COF_NAME, SUP_ID, PRICE, SALES, TOTAL from COFFEES";
    try (Statement stat = con.createStatement()) {
        ResultSet rs = stat.executeQuery(query);
        while (rs.next()) {
            String coffeeName = rs.getString("COF_NAME");
            int supplierID = rs.getInt("SUP_ID");
            float price = rs.getFloat("PRICE");
            int sales = rs.getInt("SALES");
            int total = rs.getInt("TOTAL");
            System.out.println(coffeeName + ", " + supplierID + ", " + price +
                ", " + sales + ", " + total);
        }
    } catch (SQLException e) {
        JBCTutorialUtilities.printSQLException(e);
    }
}
```

Separating API and Driver: we can easily switch to another DB without affecting the client code

- 通过 DriverManager/驱动建立 Connection（可根据不同 DBMS 构造不同的 URL）
- 使用 Connection 创建 Statement/PreparedStatement 并执行 SQL 查询
- 通过 ResultSet 逐行读取结果并将列值取出到 Java 变量中
- 优点：JDBC 将 API 与具体驱动分离，可以在不修改客户端业务代码的情况下切换底层数据库实现
- 缺点与挑战：
 - 需要手写大量 SQL 查询语句
 - 需手动在 Java 对象与关系型表/字段之间进行映射（取值、赋值、类型转换）
 - 当对象模型复杂时（嵌套、关联、继承等），映射工作容易变得繁琐且出错
- 对象模型与关系模型不匹配的典型问题：
 - 颗粒度（Granularity）：对象模型可有多层次结构，但数据库只有表和列（例如 Person 与 Address 可能是两个类但存在同一表中）
 - 继承（Inheritance）：对象支持继承关系，数据库表本身不直接支持继承
 - 标识（Identity）：数据库用主键标识行；Java 同时有对象身份（==）和相等性（equals）
 - 关联（Associations）：Java 使用引用表示双向关联；数据库使用单向的外键来表示关联
 - 数据导航（Data Navigation）：在 Java 中通过对象图遍历引用获取关联对象；在关系型 DB 中通常通过 SQL join 来检索关联数据
- 对象-关系映射（ORM）简介：

```

List<Student> students = new ArrayList<>();

String sql = "SELECT id, name FROM student";

try (Connection conn = DriverManager.getConnection(url, user, pass);
    PreparedStatement stmt = conn.prepareStatement(sql);
    ResultSet rs = stmt.executeQuery()) {

    while (rs.next()) {
        Student s = new Student();
        s.setId(rs.getInt("id"));
        s.setName(rs.getString("name"));
        students.add(s);
    }

} catch (SQLException e) {
    e.printStackTrace();
}

```

With plain JDBC

- ORM 库/技术允许我们以面向对象的方式查询与操作数据库数据
- 使用 ORM 时通常不直接编写 SQL，而是直接操作 Java 对象（例如 repository.findAll() 返回对象列表）
- 优势：减少手写 SQL 与手工映射，提高开发效率，代码更贴近对象范式
- 注意：ORM 并非万能，复杂查询或性能调优时仍可能需要手写 SQL 或理解底层生成的语句

• Java Persistence API (JPA)

• JPA (Java 持久化 API)

- Java EE 中针对对象关系映射（ORM）的标准规范
- 参考实现：EclipseLink（GlassFish 使用）
- 其他实现：Hibernate、Apache OpenJPA

• JPA 与具体实现（如 Hibernate）的类比

<pre> public interface JPA { public void insert(Object obj); public void update(Object obj); public void delete(Object obj); public Object select(); } http://tothought.com/post/2 </pre>	<pre> public class Hibernate implements JPA { public void insert(Object obj) { //Persistence code } public void update(Object obj) { //Persistence code } public void delete(Object obj) { //Persistence code } } </pre>
---	--


```

public class MyApplication {

    public static JPA jpa = new Hibernate();

    public static void main(String[] args) {
        Object object = new Object();
        jpa.insert(object); //writes to DB
    }

}

```

We could switch to other JPA implementations easily

- JPA 定义接口/规范（插入、更新、删除、查询等方法）
- Hibernate 等实现类实现这些接口，封装持久化细节
- 好处：可以在不改业务代码的情况下切换不同的 JPA 实现

- Hibernate 架构要点

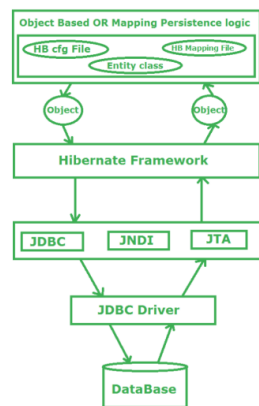


Fig: Working Flow of Hibernate framework to save/retrieve the data from the database in form of Object

- 持久化逻辑：配置（如何连接数据库）与映射（如何将类映射到表）
- 实体类：普通的 Java 类（表示数据库中的实体）
- 客户端通过面向对象的语法操作实体对象（如 setName(...)），由 Hibernate 框架在底层处理持久化
- Hibernate 通过 JDBC、JNDI、JTA 等与数据库交互，最终依赖 JDBC 驱动进行实际通信